40 Days of LeetCode Challenge

40 days, 40 problems
DAY :9

LEETCODE PROBLEM NO. **1672 : Richest Customer Wealth**

Problem Link : https://leetcode.com/problems/richest-customer-wealth/description/
Easy · 2D Arrays · Nested Loops ·
STEP BY STEP SOLUTION

Step 1. Understand the question :-

We're given a 2D integer array called accounts.

- Each row represents one customer.
- Each column represents a bank account belonging to that customer.
- Each element tells us how much money that customer has in that particular bank.

A customer's wealth is the sum of all the values in their row.

Our task is to return the maximum row sum.

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: accounts = [[1,2,3],[3,2,1]]
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.

Customer 1			
1	**2**		**3**
Total wealth: 1 + 2 + 3		6	
Customer 2			
3	**2**		**1**
Total wealth: 3 + 2 + 1		6	
Richest customer's wealth			6

Example 2:
Input: accounts = [[1,5],[7,3],[3,5]]
Output: 10
Explanation:
1st customer has wealth = 6
2nd customer has wealth = 10
3rd customer has wealth = 8
The 2nd customer is the richest with a wealth of 10.

Customer 1	
1	**5**
Total wealth: 1 + 5	6
Customer 2	
7	**3**
Total wealth: 7 + 3	10
Customer 3	
3	**5**
Total wealth: 3 + 5	8
Richest customer's wealth	10

Example 3:
Input: accounts = [[2,8,7],[7,1,3],[1,9,5]]
Output: 17
Explanation:
1st customer has wealth = 17
2nd customer has wealth = 11
3rd customer has wealth = 15
The 1st customer is the richest with a wealth of 17.

Customer 1		
2	**8**	**7**
Total wealth: 2 + 8 + 7	17	
Customer 2		
7	**1**	**3**
Total wealth: 7 + 1 + 3	11	
Customer 3		
1	**9**	**5**
Total wealth: 1 + 9 + 5	15	
Richest customer's wealth		17

Step 3. Identify the pattern

The main concepts are:

- 2D array traversal: visit each row and its elements.
- Running sum: Adding each customer's account balances.
- Maximum tracking: compare each customer's total with the highest wealth found so far.

The logic is essentially: $$\text{richestWealth} = \max_i \left(\sum_j accounts[i][j] \right)$$

In simple words: add each row, then keep the biggest sum.

Step 4. Build the logic

Here's the approach we'll follow.

We'll maintain two variables:

- wealth: calculates the current customer's total.
- maxWealth: stores the highest total found so far.

Algorithm

We'll follow your code exactly.

1. Initialize maxWealth = 0 to store the greatest wealth found so far.
2. Use an enhanced for loop to visit each customer's row.
3. Set wealth = 0 for the current customer.
4. Use another enhanced for loop to add all the money in that customer's accounts.
5. Update maxWealth using Math.max(maxWealth, wealth).
6. After checking every customer, return maxWealth.

Pseudocode

```
START

  maxWealth = 0

  FOR each customer in accounts

    wealth = 0

    FOR each money in customer

      wealth = wealth + money

    maxWealth = maximum(maxWealth, wealth)

  RETURN maxWealth

END
```

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 1672.

```
class Solution {

    public int maximumWealth(int[][] accounts) {

        int maxWealth = 0;



        for (int[] customer : accounts) {

            int wealth = 0;

            for (int money : customer) {

                wealth += money; }

            maxWealth = Math.max(maxWealth, wealth);

        } return maxWealth;

    }

}
```

How to submit

1. Open LeetCode Problem 1672. Richest Customer Wealth
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. *class Solution {*

Defines the class expected by LeetCode.

B. *public int maximumWealth(int[][] accounts) {*

Defines a method that accepts a 2D integer array and returns an integer representing the maximum wealth.

C. *int maxWealth = 0;*

Initializes our maximum wealth tracker. Whenever we discover a customer with a greater total, this value will be upd
ated.

D. *for (int[] customer : accounts) {*

int wealth = 0;

- int[] customer represents one complete row of the 2D array.
- The outer loop processes one customer at a time.
- wealth = 0 resets the running total for each new customer.

E. for *(int money : customer) {*
wealth += money;
}

The inner loop visits every account balance belonging to the current customer.

wealth += money is shorthand for:

wealth = wealth + money;

Each account balance gets added to the customer's running total.

F. *maxWealth = Math.max(maxWealth, wealth);*
}

Math.max(a, b) returns whichever value is larger.

For example, if maxWealth = 11 and the current customer's wealth = 17, then:

Math.max(11, 17) // returns 17
And then closes the outer loop after all customers have been processed

Our maximum is now updated to 17.

G. *return maxWealth;*

Returns the richest customer's total wealth.

7. Complete dry run

Let's dry-run Example 3.

Input:

```
accounts = new int[][] {
   {2, 8, 7},
   {7, 1, 3},
   {1, 9, 5}
};
```

Initially, maxWealth = 0.

Customer 1		*row 1*
wealth = 0 → 2 → 10 → 17		
After adding 2, 8 and 7, wealth = 17.		
maxWealth = Math.max(0, 17)	17	
Customer 2		*row 2*
wealth = 0 → 7 → 8 → 11		
The current wealth is 11, which is lower than 17.		
maxWealth = Math.max(17, 11)	17	
Customer 3		*row 3*
wealth = 0 → 1 → 10 → 15		
The current wealth is 15, still lower than 17.		
maxWealth = Math.max(17, 15)		17
Final answer		**17**

Step 7. Time complexity & Auxiliary space complexity

Let:

- (m) = number of customers (rows).
- (n) = number of bank accounts per customer (columns).

Time complexity: (O(m x n))

We visit every account balance exactly once. The outer loop processes customers, and the inner loop processes th eir accounts.

Auxiliary space complexity: (O(1))

We only use maxWealth, wealth, customer, and money as working variables. No extra array or collection is created.

NOTE:-

Can we optimize it further?

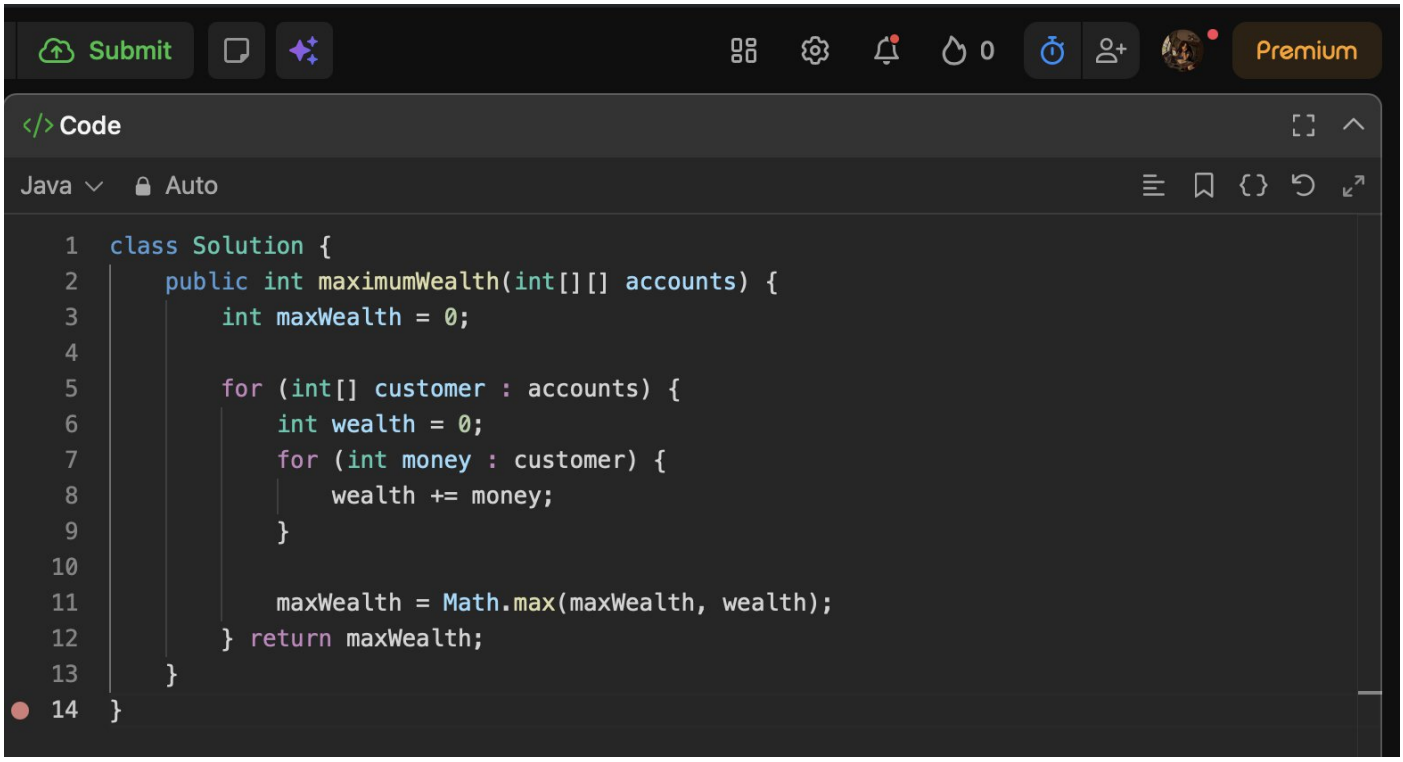
Your solution is already asymptotically optimal.

- Every account balance may affect the answer, so we need to examine all the values.
- The algorithm takes (O(m x n)) time, which is optimal for this problem.
- We already use (O(1)) auxiliary space.

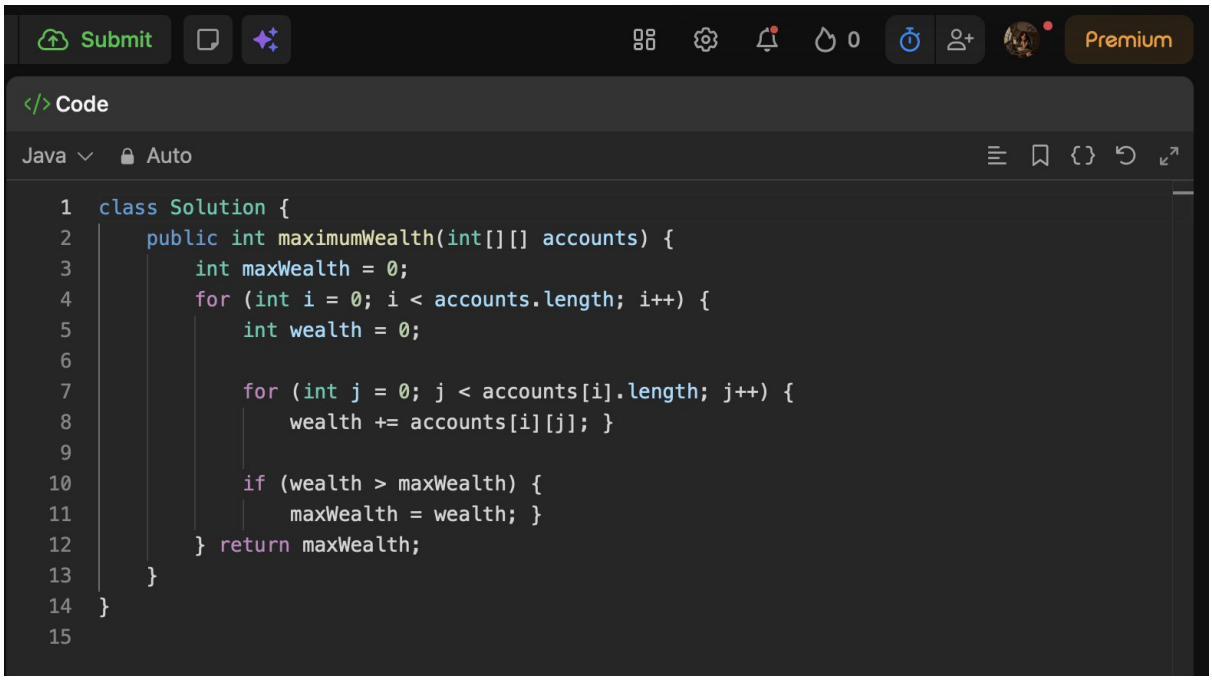
- We can write an indexed for loop instead, or use an if statement instead of Match.max(), but neither change improves the time and space complexity.

```
class Solution {
  public int maximumWealth(int[][] accounts) {
    int maxWealth = 0;
    for (int i = 0; i < accounts.length; i++) {
      int wealth = 0;
      for (int j = 0; j < accounts[i].length; j++) {
        wealth += accounts[i][j]; }
      if (wealth > maxWealth) {
        maxWealth = wealth;
      }
    } return maxWealth;
  }
}
```

SCREENSHOT:-



OR


Submit
0
Premium
Code
Java
Auto
class Solution {
public int maximumWealth(int[][] accounts) {
int maxWealth = 0;
for (int i = 0; i < accounts.length; i++) {
int wealth = 0;
for (int j = 0; j < accounts[i].length; j++) {
wealth += accounts[i][j]; }
if (wealth > maxWealth) {
maxWealth = wealth; }
} return maxWealth;
}
}